

Seguridad

¿Cuáles son las tres categorías principales de permiso frente a un archivo? (owner, group, universo, ACL, capability)

Owner: Usuario propietario del archivo. Este tiene permisos específicos sobre el archivo como leer, escribir, ejecutar.

Group: Grupo de usuarios que tienen permisos específicos sobre el archivo como por ejemplo solo lectura y ejecución a diferencia del owner.

Others: Usuarios que no son propietarios ni miembros del grupo asignado. Determinan lo que el resto del sistema puede hacer con el archivo.

ACL: (Access Control List). Posibilitan los permisos de usuarios específicos a más de un grupo. Puedo crear una lista con múltiples usuarios o grupos.

Capability: Manera de otorgar permisos específicos relacionados con las capacidades del sistema a un binario o proceso en vez de otorgar permisos específicos como root. Ej: Abrir sockets de una red sin ser root.

Explicar cómo son las ACLs en UNIX.

Cada archivo puede tener una ACL con múltiples usuarios/grupos. Podemos setear que el propietario puede leer o escribir. Que un usuario predeterminado solo puede leer, que un grupo solo puede ejecutar, etc.

Explicar SETUID. Explicar otro método (distinto a setuid y setgid) para ejecutar con permisos de otro usuario.

SETUID es un permiso de acceso que pueden asignar a archivos o directorios en un sistema operativo basado en Unix. Se utiliza para permitir que un usuario ejecute un binario con privilegios elevados temporalmente para realizar una tarea específica. Si un archivo tiene activado el bit "SETUID" se identifica con una s en el listado.

Otra forma de ejecutar con permisos de otro usuario es usar *sudo*. Permite ejecutar, si tenemos la contraseña root, un binario como si fuéramos root.

Explicar cómo funcionan los canarios.

Los canarios son unas variables aleatorias que pone el compilador entre el prologo y las variables/argumentos del stack de una funcion (hay que especificar que queremos compilar con canarios) luego cuando se esta por salir de la funcion se chequea que el valor del canario sea igual a cuando se inicio la ejecucion. Si no es igual implica que hubo un buffer overflow (alguien piso el stack para tal vez cambiar la direccion de retorno) y entonces se levanta una excepcion.

¿Para qué sirve el bit NX (No eXecute) de la tabla de páginas?

Explicar buffer overflow y mecanismos para prevenirlo. (canario, páginas no ejecutables, randomización de direcciones) ¿En qué parte de la memoria se buscan hacer los buffer overflow? (En el stack, pero también se puede hacer en el heap).

Un buffer overflow se genera cuando en una funcion se declara un buffer de datos limitados que se guarda en el stack y una funcion escribe mas bytes que el limite del buffer, sobrescribiendo asi sobre otros datos del stack como punteros, variables locales y hasta la direccion de retorno. Un actor malicioso podria usar esta vulnerabilidad para cambiar los datos del stack, como la direccion de retorno, para manipular el flujo de ejecucion del programa.

Se puede prevenir con stack canaries como se explico antes. Otra forma es ASLR donde se modifica de manera aleatoria la direccion base de regiones importantes de memoria.

La ultima forma que se ve en la materia es DEP donde algunas regiones de memoria se marcan como data only. No se permiten que codigo ejecutable se asigne en ese area. Se implementa usando el bit NX en intel como se pregunta mas arriba.

Los buffer overflows se buscan hacer en el stack por la direccion de retornno. En el heap creo es medio complicado porque el kernel te asigna paginas que estan disponibles y no se si se puede predecir el valor de algunas secciones como para modificarlas a voluntad.

¿Cómo se genera el canario? ¿En tiempo de compilación o de ejecución?

Si compilamos con una flag especifica el compilador pondra un valor random como canario en el stack entre el prologo y las variables locales.

Explicar el ataque Return to libc.

El objetivo es redirigir el return a funciones de libc que generalmente se cargan en memoria como una shared library para los procesos. El atacante redirige el return

mediante un buffer overflow a funciones como `system()`, `execve()` o `exit()`.

No se inyecta código, sino que se llaman funciones de `libc` con argumentos. Esto implica que mecanismos de protección como el bit NX NO FUNCIONAN. Ojo, las funciones de `libc` no tienen privilegios elevados.

Dar un ejemplo de un error de seguridad ocasionado por race condition. (cambiar concurrentemente puntero de symbolic links)

1. Creamos un archivo en el directorio `tmp` de linux.
2. Creamos un soft link al archivo usando su path `"/tmp/archivo"`
3. Ejecutamos el programa con privilegios elevados que verifica los permisos del archivo apuntado por el soft link.
4. Justo después de que pasa la verificación pero antes de que se abra el archivo cambiamos el soft link para que apunte a un archivo crítico del sistema como `"/etc/passwd"`

```
ln -sf /etc/passwd /tmp/archivo
```

5. Podemos agregar un nuevo usuario con privilegios elevados.

Explicar las diferencias entre MAC y DAC.

En **DAC** los atributos de seguridad se tienen que definir explícitamente por el dueño, este decide los permisos. En **MAC** los atributos son controlados por el sistema operativo, tiene más presencia en sistemas de alta delicadeza con respecto a la confidencialidad.

¿Qué es una firma digital? Explicar cómo es el mecanismo de autenticación.

Es una forma de autenticación criptográfica utilizada para verificar la integridad y autenticidad de un mensaje o documento.

1. Se usa una función de hash para obtener una "variable" que representa al documento.
2. Creamos una "variable" llamada *firma digital* que se calcula modificando el hash anterior con la clave privada del emisor. La única forma de deshacer esta modificación es con la clave pública del emisor.
3. Enviamos el documento original y la *firma digital*.
4. El receptor ya tenía la clave pública así que simplemente calcula el hash del documento, deshace la *firma digital* con la clave pública y verifica que ambos hashes son iguales. O sea, que el documento recibido es el mismo que firmó el emisor.

¿Qué es una función de hash? ¿Cómo se usa en el log-in de un SO? ¿Qué problemas tiene usar un hash para autenticar usuarios? ¿Cómo se puede mejorar la seguridad? ¿Qué es un SALT? ¿Cómo podemos hacer que calcular la función de hash sea más costoso? ¿Qué otros usos tiene la función de hash en seguridad?

Es un algoritmo que toma un conjunto de datos y genera un valor de longitud fija.

Se usa para almacenar contraseñas de manera segura. En vez de usar texto claro el ssos las hashea.

El atacante puede probar todas las combinaciones para obtener el hash que coincida. Si la función es débil (hay muchas colisiones) se pueden generar dos hashes que mapean a la misma contraseña reduciendo el tiempo de fuerza bruta.

Usar funciones lentas para que sea más costoso a los atacantes usar fuerza bruta o usar un SALT (valor aleatorio que se añade a la contraseña antes de aplicar la función de hash) garantiza que aunque dos usuarios tengan la misma contraseña sus hashes serán distintos.

Usamos funciones lentas o varias iteraciones de hash.

Verificar que los datos enviados y recibidos no sufrieron cambios en el camino por un tercer actor por ejemplo.

¿Qué es una clave pública/privada? ¿Cómo podemos distribuir una clave pública?

Son claves que se generan mediante un algoritmo para un actor. De esta manera si distribuimos la clave pública podemos firmar el hash de documentos con nuestra clave privada y que el receptor que recibe el documento y el hash firmado decifre con la clave pública el hash firmado y lo compare con el hash del documento recibido verificando así la integridad de la información.

Es esencial poder identificar una clave pública con un actor verificado. Como por ejemplo por canales públicos masivos o por redes seguras.

(La encriptación es lo contrario a la firma digital, es el proceso inverso. En encriptación de mensajes privados nosotros ciframos con la pública y solo la persona con la privada puede decifrarlo)

Caso de uso normal: (Confidencialidad garantizada)

1. Bob genera una clave pública y una clave privada.
2. Bob le envía su **clave pública** a Alice mediante un **canal seguro** (por ejemplo, puede ser una red privada o mediante un certificado digital confiable).
3. Alice encripta el mensaje con la clave pública de Bob.
4. Alice envía el mensaje encriptado a Bob.
5. Bob desencripta el mensaje con su clave privada.

Caso problemático: (Ataque Man-in-the-Middle)

1. Bob genera una clave pública y una clave privada.
2. Bob le envía su **clave pública** a Alice mediante un **canal inseguro** (por ejemplo, correo electrónico o cualquier medio que podría ser interceptado).
3. Cristina, una atacante maliciosa, intercepta la clave pública de Bob.
4. Cristina genera su propia clave pública y privada y **le envía su clave pública** a Alice, haciéndose pasar por Bob.
5. Alice, al recibir la clave pública de Cristina (creyendo que es la de Bob), **cifra el mensaje** usando la clave pública de Cristina en lugar de la de Bob.
6. Cristina, que tiene la **clave privada** correspondiente a la clave pública que Alice usó, puede **desencriptar** el mensaje.
7. Cristina **modifica el mensaje** (por ejemplo, puede ponerle "cosas feas" a Bob o alterar el contenido de cualquier manera) y luego **cifra nuevamente el mensaje** con la clave pública de Bob.
8. Cristina envía el mensaje encriptado a Bob.
9. Bob, al recibir el mensaje, **lo desencripta** con su clave privada y **lee el mensaje modificado** sin saber que ha sido alterado.

¿Por qué es más seguro utilizar un esquema de clave pública/privada para una conexión por ssh, comparado con usuario/contraseña?

Porque las contraseñas pueden ser mas faciles de romper mediante fuerza bruta usando ingenieria social. Las claves publicas/privadas se basan en algoritmos criptograficos (mas cool).

La contraseña debe ser enviada y puede ser grabada con un keylogger mientras que la clave privada se mantiene segregada del usuario en general. Son locales, no se envian por la red.

En general los problemas son humanos. Phishing, keylogger, contraseñas adivinables, intervencion humana, etc.

¿Se puede considerar Deadlock un problema de seguridad?

Si porque rompe el principio de disponibilidad. Un deadlock puede trabar infinitamente un servicio haciendolo inaccesible para su uso. Como un servidor, una maquina, una base de datos. Es un problema grave de seguridad. Este bloqueo puede ser explotado por un atacante para interrumpir un servicio, lo que lo convierte en una vulnerabilidad que puede ser aprovechada para llevar a cabo un ataque de denegación de servicio(DoS).